RotaGrafo: Roteamento e Carga de Veículos em Centros Logísticos com Grafos

Zaine[1], Bruna[1], Kethellen[1], Rafael[1], Filipe Quaresma Pereira[1], Vitor[1]

[1]Instituição – Curso – Cidade, UF – Brasil

Abstract. *This work presents RotaGrafo, a tool for planning delivery routes from a logistics center. The delivery network is modeled as a weighted graph whose vertices are the depot and the customers, and whose edges carry distance and travel time. Routes are built with the Nearest Neighbor and Clarke-Wright heuristics and refined with 2-opt, respecting vehicle capacity, time windows and delivery priority. A web interface with an interactive map allows configuring the fleet, visualizing the routes and comparing algorithms.*

Resumo. *Este trabalho apresenta o RotaGrafo, uma ferramenta para o planejamento de rotas de entrega a partir de um centro logístico. A rede de entregas é modelada como um grafo ponderado cujos vértices são o depósito e os clientes e cujas arestas carregam distância e tempo de deslocamento. As rotas são construídas com as heurísticas do Vizinho Mais Próximo e de Clarke-Wright e refinadas com 2-opt, respeitando a capacidade dos veículos, as janelas de tempo e a prioridade das entregas. Uma interface web com mapa interativo permite configurar a frota, visualizar as rotas e comparar os algoritmos.*

1. Introdução

A distribuição de mercadorias é uma das etapas mais custosas da cadeia logística. Em um centro de distribuição urbano, uma frota limitada de veículos precisa atender dezenas ou centenas de clientes por dia. Cada veículo tem uma capacidade de carga máxima, cada cliente tem um horário em que pode receber a entrega e alguns pedidos são mais urgentes que outros. Decidir quais clientes cada veículo atende, e em que ordem, afeta diretamente a distância percorrida, o consumo de combustível, o número de veículos necessários e a quantidade de entregas atrasadas.

Esse cenário corresponde ao *Problema de Roteamento de Veículos* (VRP), em especial às variantes com restrição de capacidade (CVRP) e com janelas de tempo (VRPTW) [Toth and Vigo 2014, Solomon 1987]. O VRP generaliza o Problema do Caixeiro Viajante e é NP-difícil. Por isso, instâncias de tamanho realista são resolvidas na prática com heurísticas, que encontram boas soluções em tempo polinomial sem garantia de otimalidade.

A estrutura central do problema é um grafo: os locais de entrega são vértices e os deslocamentos entre eles são arestas ponderadas. Este trabalho desenvolve o **RotaGrafo**, uma ferramenta que:

- modela a rede de entregas como um grafo ponderado implementado do zero, sem bibliotecas prontas;
- calcula caminhos mínimos entre os pontos com o algoritmo de Dijkstra [Dijkstra 1959];

- constrói rotas com as heurísticas do Vizinho Mais Próximo e de Clarke-Wright [Clarke and Wright 1964] e as refina com 2-opt [Croes 1958];
- apresenta os resultados em uma interface web com mapa interativo, métricas e comparação entre algoritmos.

O restante do artigo está organizado da seguinte forma: a Seção 2 apresenta a modelagem do grafo e os dados utilizados; a Seção 3 descreve os algoritmos e suas complexidades; a Seção 4 mostra a arquitetura da ferramenta e a interface; a Seção 5 analisa os resultados e o desempenho; a Seção 6 discute as limitações; e a Seção 7 conclui o trabalho.

2. Modelagem do Problema

2.1. Definição formal

2.2. Diagramas UML

2.3. Base de dados

3. Algoritmos

3.1. Caminhos mínimos: Dijkstra

3.2. Vizinho Mais Próximo

3.3. Clarke-Wright (*Savings*)

3.4. Melhoria local: 2-opt

3.5. Resumo das complexidades

Tabela 1. Complexidade dos algoritmos implementados (n = número de vértices).

Algoritmo	Tempo	Espaço
Dijkstra (heap binário)	$O((V + E) \log V)$	$O(V)$
Vizinho Mais Próximo	$O(n^2)$	$O(n)$
Clarke-Wright	$O(n^2 \log n)$	$O(n^2)$
2-opt (por iteração)	$O(n^2)$	$O(n)$

4. Arquitetura e Interface

4.1. Arquitetura

A ferramenta segue uma arquitetura cliente-servidor em três camadas (Figura 1):

1. **Núcleo de grafos** (Python): a estrutura do grafo, a leitura dos dados e os algoritmos de roteirização, implementados sem bibliotecas de grafos;
2. **API REST** (Flask): expõe o núcleo por meio de rotas HTTP com respostas em JSON e controla o acesso por login e sessão;
3. **Interface web** (HTML, CSS e JavaScript com Leaflet): consome a API e apresenta os resultados sobre um mapa do OpenStreetMap.

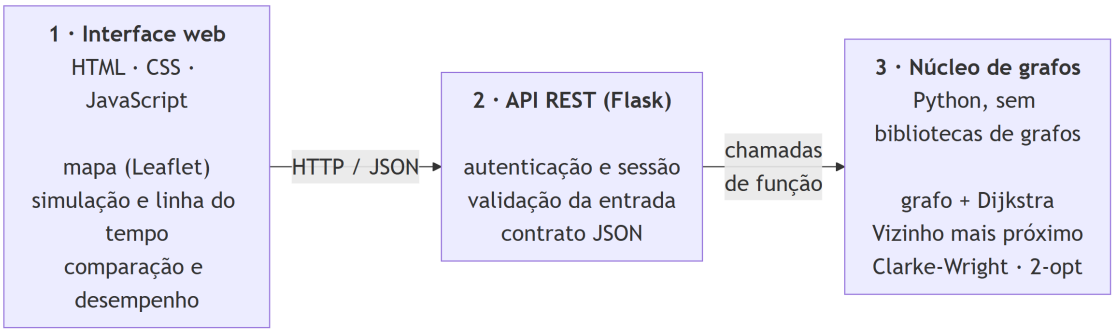


Figura 1. Arquitetura do RotaGrafo em três camadas.

A comunicação entre as camadas segue um contrato JSON fixo (Tabela 2). Esse contrato permitiu que as três duplas trabalhassem em paralelo: enquanto o núcleo era implementado, a interface foi construída sobre dados de exemplo no mesmo formato e depois conectada ao código real trocando apenas as quatro linhas do ponto de integração em `src/api/app.py`.

Tabela 2. Rotas da API.

Rota	Descrição
`GET /api/grafo`	Vértices (posição, demanda, janela, prioridade) e arestas (distância, tempo).
`GET /api/algoritmos`	Algoritmos disponíveis.
`POST /api/rotas`	Recebe frota, capacidade, algoritmo e uso do 2-opt; devolve as rotas, os horários de chegada, os clientes não atendidos, as métricas e o tempo de execução.
`GET /api/desempenho`	Recebe uma lista de tamanhos de grafo e devolve o tempo de execução de cada algoritmo em cada tamanho.
`POST /api/auth/login`	Recebe e-mail e senha e inicia a sessão.
`POST /api/auth/registrar`	Cria uma conta e inicia a sessão.
`POST /api/auth/logout`	Encerra a sessão.

4.2. Validação do contrato

Para garantir que a integração entre as partes não quebrasse a interface, foi criado um validador (`src/api/contrato.py`) com testes automatizados. Ele verifica, para várias combinações de frota, capacidade e algoritmo, que:

- toda rota começa e termina no depósito;
- nenhuma rota excede a capacidade do veículo;
- nenhum cliente é visitado mais de uma vez;
- todo cliente aparece em exatamente uma rota ou na lista de não atendidos;
- o número de rotas não passa do número de veículos disponíveis.

4.3. Testes de corretude

Os testes de contrato conferem o formato das respostas, mas não se os valores estão certos. Por isso, um segundo conjunto de testes (`tests/test_corretude.py`) recalcula os resultados de forma independente. Ele implementa uma versão de referência do Dijkstra, conferida contra o algoritmo de Floyd-Warshall, e a usa para verificar, em várias instâncias e configurações de frota, que:

- o grafo é conexo a partir do depósito e não tem laços nem arestas repetidas;
- a distância de cada rota é igual à soma dos caminhos mínimos entre paradas consecutivas;
- nenhum veículo chega a um cliente antes do que o caminho mínimo em tempo permite;
- cada entrega começa em $\max(\text{chegada}, a_i)$, ou seja, o veículo espera a janela abrir, mas não espera sem necessidade, e é marcada como atrasada exatamente quando começa depois de b_i;
- as métricas totais são a soma das rotas e a ocupação é carga$/Q$;
- com frota suficiente, nenhum cliente fica sem atendimento;
- em instâncias pequenas (5 a 7 clientes), nenhuma solução é mais curta que o ciclo ótimo calculado por força bruta, o que denunciaria um erro no cálculo das distâncias.

Dois testes adicionais avaliam a qualidade das heurísticas: o 2-opt nunca deve aumentar a distância total, e o Clarke-Wright com 2-opt deve ficar a no máximo 25% do ótimo nas instâncias pequenas.

4.4. Controle de acesso

O acesso à ferramenta exige login. Sem uma sessão válida, as páginas redirecionam para a tela de login e as rotas da API respondem com o código HTTP 401. As senhas nunca são armazenadas: o servidor guarda apenas o *hash* gerado pela biblioteca Werkzeug. A sessão fica em um *cookie* `HttpOnly`, que pode durar 30 dias quando o usuário marca a opção "Manter conectado". Para dificultar ataques de força bruta, cinco tentativas erradas seguidas para o mesmo e-mail e endereço IP bloqueiam o login por 15 minutos, e cada endereço IP pode criar no máximo 10 contas por hora.

4.5. Interface

A interface (Figura **??**) é dividida em um painel lateral com três abas (Rotas, Comparação e Desempenho), um mapa e uma linha do tempo. Suas funcionalidades são:

- **Cenário**: número de vértices e arestas, demanda total, clientes de alta prioridade e uma estimativa do número mínimo de veículos ($\lceil \sum q_i/Q \rceil$). As arestas do grafo podem ser exibidas sobre o mapa;
- **Configuração**: número de veículos, capacidade, algoritmo e uso do 2-opt;
- **Rotas**: cada veículo é desenhado com uma cor, e as cores seguem uma paleta validada para daltonismo. Cada rota tem um cartão com distância, tempo, ocupação da carga e lista de entregas. Clicar em uma rota a destaca no mapa e na linha do tempo;
- **Simulação do dia**: um controle de reprodução anima os veículos percorrendo as rotas conforme os horários calculados. Os clientes mudam de estado (aguardando, em atendimento, entregue no prazo, entregue com atraso) e uma barra permite ir a qualquer horário (Figura **??**);

- **Linha do tempo**: um diagrama de Gantt com uma linha por veículo mostra as viagens, as esperas pela abertura da janela e as entregas. Entregas atrasadas são destacadas e, ao passar o mouse sobre uma entrega, a janela de tempo do cliente aparece ao fundo. Assim, as restrições de tempo do VRPTW ficam visíveis de forma direta;
- **Comparação**: executa todas as combinações de algoritmo, com e sem 2-opt, e mostra uma tabela e um gráfico de barras com a melhor configuração destacada (Figura **??**);
- **Desempenho**: mede o tempo de execução dos algoritmos para grafos de vários tamanhos e o exibe em um gráfico de linhas, o que permite confrontar o comportamento empírico com a complexidade teórica (Seção 5);
- **Guia de uso**: um tour interativo, exibido no primeiro acesso, destaca cada parte da tela e explica como usá-la;
- **Exportação**: salva o resultado em JSON para análise posterior.

Os diagramas de componentes, de sequência e de estados da interface estão nas Figuras 2, 3 e 4.

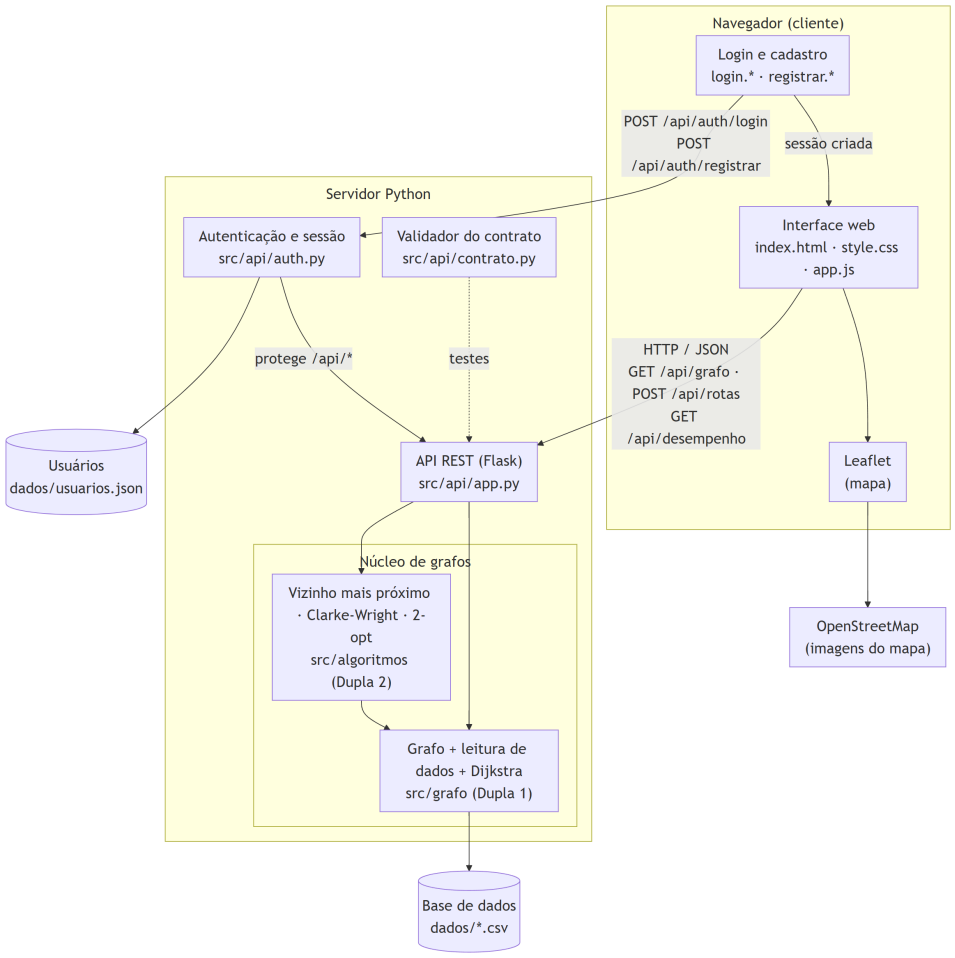


Figura 2. Diagrama de componentes do RotaGrafo.

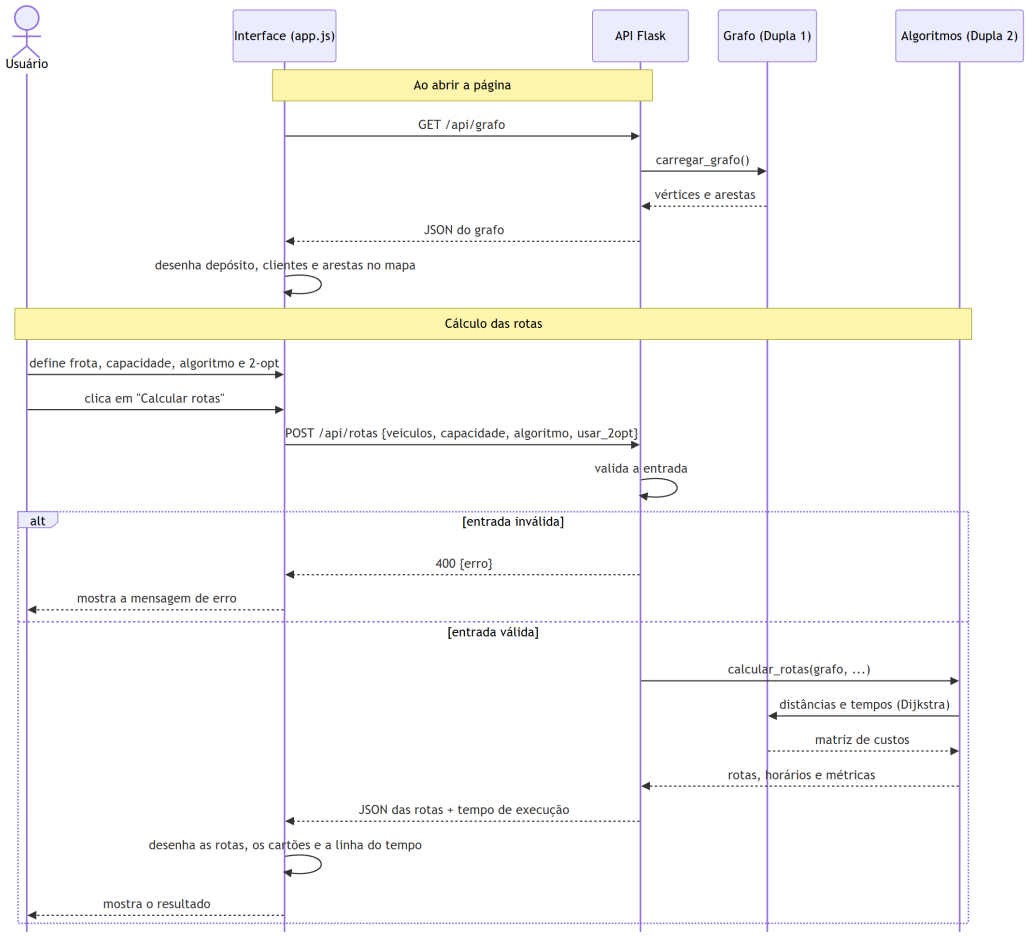


Figura 3. Diagrama de sequência do cálculo de rotas.

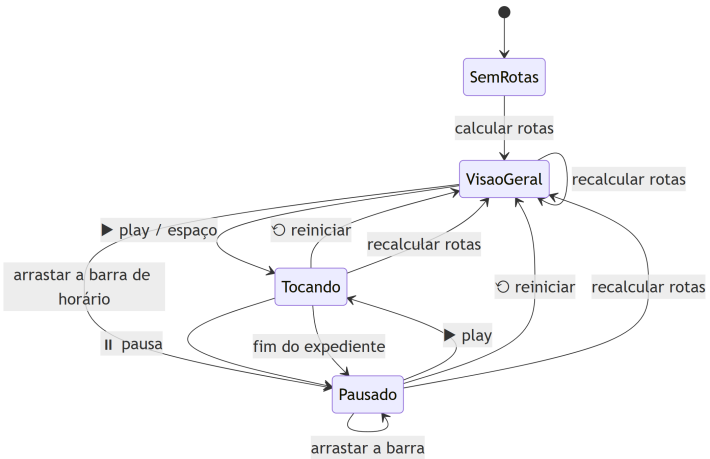


Figura 4. Diagrama de estados da simulação.

5. Resultados e Avaliação de Desempenho

6. Limitações

7. Conclusão

8. Divisão de Responsabilidades

Tabela 3. Responsabilidades de cada membro.

Dupla	Membro	Atividades
1	Zaine	TODO
1	Bruna	TODO
2	Kethellen	TODO
2	Rafael	TODO
3	Filipe	Interface web: mapa, rotas, simulação do dia, linha do tempo e guia de uso; relatório
3	Vitor	API Flask, contrato e testes de integração, aba de desempenho, diagramas UML da arquitetura; login, cadastro de contas e controle de sessão

Referências

Clarke, G. and Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4):568–581.

Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271.

Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265.

Toth, P. and Vigo, D. (2014). *Vehicle Routing: Problems, Methods, and Applications*. SIAM, 2 edition.